

FCRTCINITIALISER

Routine d'initialisation du CRTC (Cathode Ray Tube Controller, 6845) sur Amstrad CPC. Configure la géométrie complète de l'écran : résolution visible, adresse de base VRAM, timings de synchronisation.

Code

```
; -----  
; ROUTINE : F_CRTC_32x24  
; -----  
; Entrée : Aucune  
; Sortie : Aucune  
; Modifie : AF, BC, HL (selon l'implémentation de la macro)  
; -----  
F_CRTC_INITIALISER  
    LD    HL, CRTC_TABLE      ; HL = pointeur vers les 14 valeurs à écrire  
    LD    C, 13               ; C = 13 -> point de départ (registre R13)  
    mCRTC_SCR_SETUP (VOID)    ; boucle d'écriture registre par registre  
    RET  
; SIZE : 3+2+13+1 = 19o
```

Macro sous-jacente : mCRTCSCRSETUP

```
MACRO mCRTC_SCR_SETUP  
    LD    B, #BC              ; B = 0xBC -> CRTC_REG (port sélection)  
    @BC_CRTC  
    OUT   (C), C              ; sélectionne le registre C  
    LD    B, #BE              ; port data (0xBD) -- astuce : OUTI décrémente B avant  
nt  
    OUTI                      ; DEC B, envoie (HL) sur 0xBD, INC HL  
    DEC   B                   ; B = 0xBC -> CRTC_REG à nouveau  
    DEC   C                   ; registre suivant (décroissant)  
    JP    P, @BC_CRTC         ; boucle tant que C >= 0  
MEND
```

Le double rôle de C

C sert à la fois de **numéro de registre CRTC** (via `OUT (C), C`) et de **compteur de boucle** (`DEC C` + `JP P`). En partant de `13` et en descendant jusqu'à `0` inclus (`JP P` continue tant que le résultat

reste positif ou nul), la macro écrit les **14 registres CRTC** (`R0` à `R13`) — l'ensemble complet des registres configurables du 6845.

L'astuce OUTI

`OUTI` décrémente `B` **avant** d'émettre la sortie. En préchargeant `B = #BE`, l'instruction se retrouve à écrire sur le port `#BD` (data CRTC) en une seule instruction au lieu de charger le port puis écrire séparément — économie de cycles et d'octets.

Fichier CRTC_TABLE.INC — constantes de géométrie

```
CRTC_R1 EQU 32      ; Largeur visible en caractères
CRTC_R6 EQU 24      ; Hauteur visible en lignes de caractères
CRTC_R12 EQU #30    ; Adresse de départ écran poids fort (#30 -> #C000)
CRTC_R13 EQU 00     ; Adresse de départ écran poids faible
```

Constantes dérivées, utilisées dans tout le reste du projet :

Constante	Formule	Valeur
<code>SCR_VRAM_BASE</code>	—	<code>#C000</code>
<code>SCR_WIDTH_BYTE</code>	<code>CRTC_R1*2</code>	64 (<code>#40</code>)
<code>SCR_PX_WIDTH_M0</code>	<code>CRTC_R1*4</code>	128
<code>SCR_PX_HEIGHT_M0</code>	<code>CRTC_R6*8</code>	192
<code>SCR_PX_PER_BYTE_M0</code>	—	2 (Mode o)

Table CRTC_TABLE — les 14 registres

CRTC_TABLE			
DB	CRTC_R13	; R13 - Adresse de départ écran poids faible	
DB	CRTC_R12	; R12 - Adresse de départ écran poids fort (#30 -> #C000)	
DB	0	; R11 - Ligne de fin du curseur	
DB	0	; R10 - Ligne de début du curseur (0 = caché)	
DB	7	; R9 - Hauteur d'un caractère (7 = 8 lignes de pixels)	
DB	0	; R8 - Mode de balayage (0 = pas d'entrelacement)	
DB	30	; R7 - Position de la synchronisation verticale	
DB	CRTC_R6	; R6 - Hauteur visible (24 lignes de caractères)	
DB	#00	; R5 - Ajustement vertical fin	
DB	38	; R4 - Hauteur totale de la trame	
DB	#86	; R3 - Largeur des pulses de synchro (H=14, V=9)	
DB	42	; R2 - Position de la synchronisation horizontale	
DB	CRTC_R1	; R1 - Largeur visible (32 caractères)	
DB	#3F	; R0 - Largeur totale de la ligne	
; R14/R15 curseur -- non utilisés			
; R16/R17 light pen -- lecture seule, non configurables			

Détail des registres écrits

Registre	Rôle	Valeur	Effet
R0	Largeur totale ligne	#3F (63)	Timing horizontal total, incluant le blanking
R1	Largeur visible	32	32 caractères affichés par ligne
R2	Position sync horizontale	42	Décalage du signal HSYNC
R3	Largeur pulses synchro	#86	H=14 (bits 0-3), V=9 (bits 4-7)
R4	Hauteur totale trame	38	Nombre de lignes de caractères total (incluant blanking)
R5	Ajustement vertical fin	0	Pas d'ajustement supplémentaire
R6	Hauteur visible	24	24 lignes de caractères affichées
R7	Position sync verticale	30	Décalage du signal VSYNC
R8	Mode de balayage	0	Pas d'entrelacement
R9	Hauteur d'un caractère	7	8 lignes de pixels par caractère (0-7 inclus)
R10	Début curseur	0	Curseur masqué
R11	Fin curseur	0	Curseur masqué
R12	Adresse écran (poids fort)	#30	Combiné à R13 -> base VRAM #C000
R13	Adresse écran (poids faible)	0	Combiné à R12 -> base VRAM #C000

Cohérence avec le reste du projet

Cette table confirme la géométrie 128×192 utilisée dans toutes les routines développées au fil du projet :

- `SCR_PX_WIDTH_M0 = 128` -> cohérent avec `RIGHT_LIMIT = 126` (balle), les limites de la raquette, et le stride `#40` (`SCR_WIDTH_BYTE`) utilisé dans `mXY_TO_VRAM_128x192_M0`.
- `SCR_PX_HEIGHT_M0 = 192` -> cohérent avec `BOTTOM_LIMIT = 188` (balle 4px de haut) et `PADDLE_PX_BOTTOM_LIMIT = 168` (raquette 24px de haut).
- `SCR_VRAM_BASE = #C000` -> cohérent avec le `#C0` codé en dur dans toutes les macros de calcul d'adresse (`NextLine`, `mSET_NEXTLINE_HL`, etc.).

Piste d'amélioration : dériver les constantes plutôt que les coder en dur

Les limites utilisées dans les routines de jeu (`126`, `188`, `168`, `#D050`, `#F2DF`) ont été calculées manuellement à partir de cette géométrie, mais pourraient être exprimées directement en fonction de `SCR_PX_WIDTH_M0` / `SCR_PX_HEIGHT_M0` :

```
RIGHT_LIMIT          equ SCR_PX_WIDTH_M0 - 2
BOTTOM_LIMIT         equ SCR_PX_HEIGHT_M0 - 4
PADDLE_PX_BOTTOM_LIMIT equ SCR_PX_HEIGHT_M0 - PADDLE_PX_HEIGHT_M0
```

Cette approche rendrait le projet résilient à un changement de résolution (ex: retour à 160×200 avec `CRT_C_R1=40` / `CRTC_R6=25`), en centralisant toute adaptation dans `CONSTANTES.INC` plutôt que de traquer chaque valeur codée en dur dans chaque routine.